

Date:
Student Name:
Student ID:

AI Workshop #7

Logic Programming

7.1. Introduction

In this workshop, we are going to learn how to write programs using **logic programming**.

We will discuss various programming paradigms and see how programs are constructed with logic programming.

We will learn about the building blocks of logic programming and see how to solve problems in this domain.

We will implement Python programs to build various solvers that solve a variety of problems.

By the end of this workshop, you will have a better understanding of these topics:

- What is logic programming?
- Understanding the building blocks of logic programming
- Solving problems using logic programming
- Installing Python packages
- Matching mathematical expressions
- Validating primes
- Parsing a family tree
- Analyzing geography
- Mini-project: Building a puzzle solver

Before we start logic programming in Python, we need to install a couple of packages. The package `logpy` is a Python package that enables logic programming in Python. We will also be using `SymPy` for some of the problems.

So, let's go ahead and install **kanren** (a new replacement of `logpy`, URL: <https://github.com/logpy/logpy> (<https://github.com/logpy/logpy>)) and `sympy` using `pip`:

```
pip install kanren  
pip install sympy
```

7.2 What is Logic Programming (LP)?

Logic programming is a programming paradigm, which basically means it is a way to approach programming. Before we talk about what it constitutes and how it is relevant in **Artificial Intelligence (AI)**, let's talk a bit about **programming paradigms**.

The concept of programming paradigms originates from the need to classify programming languages.

It refers to the way computer programs solve problems through code.

Some programming paradigms are primarily concerned with implications or the sequence of operations used to achieve a particular result.

Other programming paradigms are concerned about how we organize the code.

Here are some of the more popular programming paradigms:

- **Imperative:** Uses statements to change a program's state, thus allowing for side effects.
- **Functional:** Treats computation as an evaluation of mathematical functions and does not allow changing states or mutable data.
- **Declarative:** A way of programming where programs are written by describing what needs to be done and not how to do it. The logic of the underlying computation is expressed without explicitly describing the control flow.
- **Object oriented:** Groups the code within a program in such a way that each object is responsible for itself. Objects contain data and methods that specify how changes happen.
- **Procedural:** Groups the code into functions and each function is responsible for a series of steps.
- **Symbolic:** Uses a style of syntax and grammar through which the program can modify its own components by treating them as plain data.
- **Logic:** Views computation as automatic reasoning over a database of knowledge consisting of facts and rules.

Logic programming has been around for a while. A language that was quite popular during one of the last heydays of AI was **Prolog**.

It is a language that uses only three constructs:

- Facts
- Rules
- Questions

But with these three constructs you were able to build some powerful systems.

One popular usage was in the construction of "expert systems."

The idea behind was to interview human experts that had been working in a given field for a long time and codify the interview into AI systems.

Some examples of fields for which expert systems were built were:

- **Medicine** – Famous examples include MYCIN, INTERNIST-I, and CADUCEUS
- **Chemical analysis** – DENDRAL is an analysis system used to predict molecular structure
- **Finance** – Advisory programs to assist bankers in making loans
- **Debugging programs** – SAINT, MATLAB, and MACSYMA

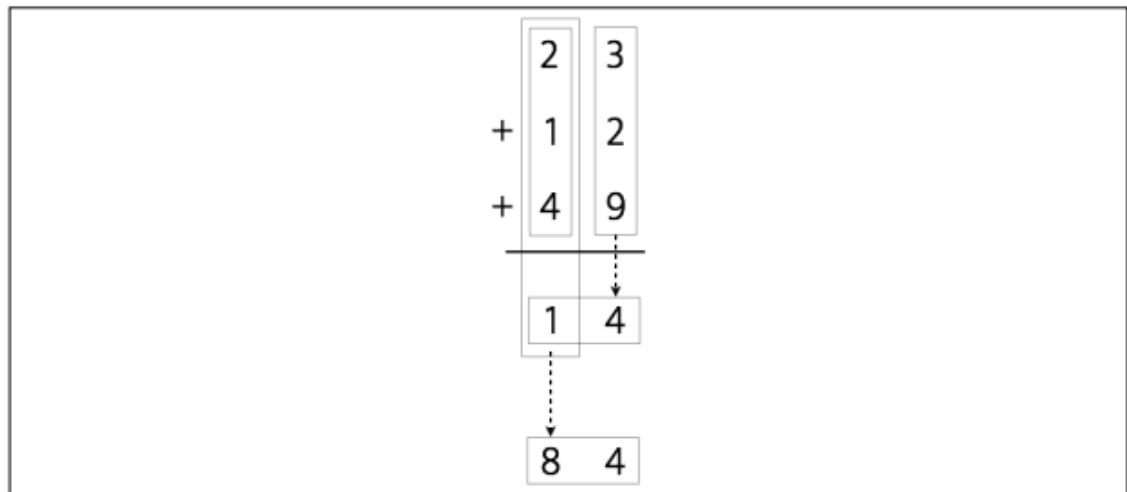
In order to understand logic programming, it's necessary to understand the concepts of computation and deduction.

To compute something, we start with an **expression** and a **set of rules**.

This set of rules is basically the program.

Expressions and rules are used to generate the output.

For example, let's say we want to compute the sum of 23, 12, and 49:



The procedure to complete the operation would be as follows:

1. Add $3 + 2 + 9 = 14$
2. We need to keep a single digit, which is the 4, and then we carry the 1
3. Add $2 + 1 + 4$ (plus the 1 we carried) $= 8$
4. Combine 8 and 4. The final result is: 84

On the other hand, to deduce something, we need to start from a conjecture.

A **proof** is constructed according to a set of rules. The process of computation is mechanical, whereas the process of deduction is more creative.

When writing a program using the logic programming paradigm, a set of statements is specified based on facts and rules about the problem domain and the solver solves it using

7.3 Solving Problems Using Logic Programming

Logic programming looks for solutions by using **facts** and **rules**.

A **goal** must be specified for each program. When a logic program and a goal don't contain any variables, the solver comes up with a tree that constitutes the search space for solving the problem and getting to the goal.

One of the most important things about logic programming is how we treat the rules.

Rules can be viewed as **logical statements**.

Let's consider the following:

Kathy orders dessert => Kathy is happy

This can be read as an implication that says: If Kathy is happy, then Kathy orders dessert.

It can also be construed as **Kathy orders dessert whenever she is happy**.

```
canfly(X) :- bird(X), not abnormal(X).
abnormal(X) :- wounded(X).
bird(john).
bird(mary).
wounded(john).
```

Here is how to interpret the rules and facts:

- John is wounded
- Mary is a bird
- John is a bird
- Wounded birds are abnormal
- Birds that are not abnormal can fly

From this, we can conclude that Mary can fly, and John cannot fly.

This construction is used in various forms throughout logic programming to solve various types of problems. Let's go ahead and see how to solve these problems in Python.

7.4 Matching mathematical expressions

We encounter mathematical operations all the time. Logic programming is an efficient way of comparing expressions and finding out unknown values.

Let's see how to do that.

1. Create a new Python file and import the following packages:

```
from kanren import run, var, fact
from kanren.assoccomm import eq_assoccomm as la
from kanren.assoccomm import commutative, associative
```

2. Define a couple of mathematical operations:

```
# Define mathematical operations
add = 'addition'
mul = 'multiplication'
```

3. Both addition and multiplication are commutative operations (meaning the operands can be flipped without changing the result). Let's specify that:

```
# Both addition and multiplication are communicative processes.
# Hence, we need to specify it and this can be done as follows
fact(commutative, mul)
fact(commutative, add)
fact(associative, mul)
fact(associative, add)
```

4. Let's define some variables:

```
# Define some variables
a, b, c = var('a'), var('b'), var('c')
```



Consider the following expression:

$$\text{expression_orig} = 3 \times (-2) + (1 + 2 \times 3) \times (-1)$$

Let's generate this expression with masked variables. The first expression would be:

$$\text{expression1} = (1 + 2 \times a) \times b + 3 \times c$$

The second expression would be:

$$\text{expression2} = c \times 3 + b \times (2 \times a + 1)$$

The third expression would be:

$$\text{expression3} = (((2 \times a) \times b) + b) + 3 \times c$$

If you observe carefully, all three expressions represent the same basic expression.

5. The goal is to match these expressions with the original expression to extract the unknown values:

```
# Generate expressions
expression_orig = (add, (mul, 3, -2), (mul, (add, 1, (mul, 2, 3)), -1))
expression1 = (add, (mul, (add, 1, (mul, 2, a)), b), (mul, 3, c))
expression2 = (add, (mul, c, 3), (mul, b, (add, (mul, 2, a), 1)))
expression3 = (add, (add, (mul, (mul, 2, a), b), b), (mul, 3, c))
```

6. Compare the expressions with the original expression. The method `run` is commonly used in **logpy**. This method takes the input arguments and runs the expression. The first argument is the number of values, the second argument is a variable, and the third argument is a function:

```
# Compare expressions
print(run(0, (a, b, c), la(expression1, expression_orig)))
print(run(0, (a, b, c), la(expression2, expression_orig)))
print(run(0, (a, b, c), la(expression3, expression_orig)))
```



Workshop 7.1 : Logic Programming on Matching Mathematical Expressions

1. Combine all the above code and named it as **expression_matcher.py**.
2. Run the code and check the results, you should get this:

```
((3, -1, -2),)
((3, -1, -2),)
()
```

3. What are the 3 values in the first two lines?
4. Why the first 2 lines have the same result while the third one return nothing?

(Hint: Using logic programming to check mathematical expression, it will check meaning and structural of the expression.)

5. In addition to **addition** and **multiplication**, also implement **subtraction** and **division** into this program.
6. Again, try to use THREE example to check the following math expression:

expression_orig: $3 / (-2) - (3 + 7 \times 3) / (-18)$

7.5 Validating primes

Let's see how to use logic programming to check for **prime numbers**.

We will use the constructs available in logpy to determine which numbers in the given list are prime, as well as finding out if a given number is a prime or not.

1. Create a new Python file and import the following packages:

```
from kanren import isvar, run, membero
from kanren.core import success, fail, goaleval, condeseq, eq, var
from sympy.ntheory.generate import prime, isprime
import itertools as it
```

2. Next, define a function that checks if the given number is prime depending on the type of data. If it's a number, then it's straightforward. If it's a variable, then we must run the sequential operation. To give a bit of background, the method `conde` is a goal constructor that provides logical AND and OR operations.

```
# Check if prime
def prime_check(x):
    if isvar(x):
        return condeseq([(eq,x,p)] for p in map(prime, it.count(1)))
    else:
        return success if isprime(x) else fail
```

3. Declare the variable x that will be used:

```
# Declare the variable
x = var()
```

4. Print first 20 prime numbers

```
# Print first 7 prime numbers
print('\nList of first 20 prime numbers:')
print((run(20,x,prime_check(x))))
```

5. If everything goes fine, you should get this:

```
List of first 7 prime numbers:
(2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67,
71)
```



Workshop 7.2 : Logic Programming on Prime Number Generator

1. Combine all the above code and named it as **prime_LP.py**.
2. Run the code and check the result of the first 20 prime numbers:
3. Save and submit the program and results.
4. Modify this program a bit so that given a list of numbers, it can extract the sublist of prime numbers.

For example:

Original list: {23, 4, 27, 17, 13, 10, 21, 29, 3, 32, 11, 19}

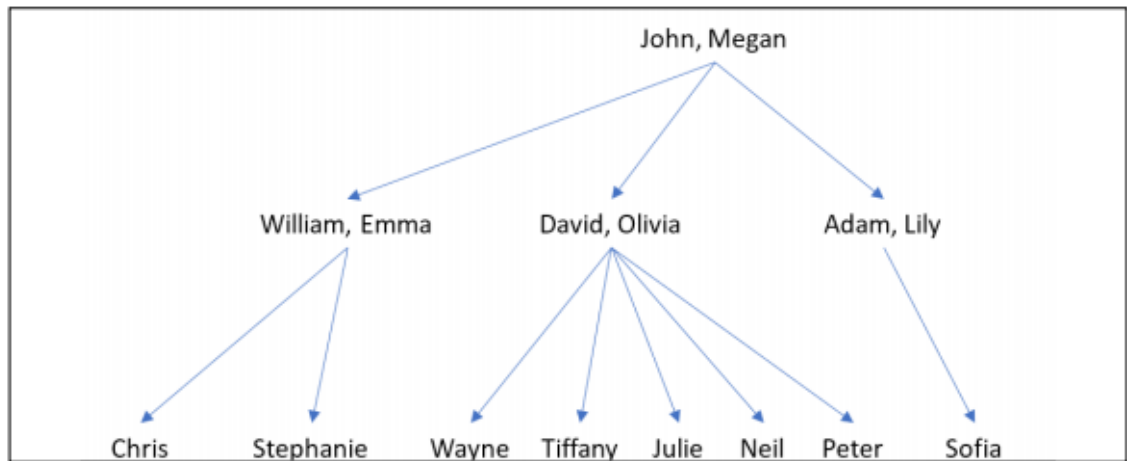
Sublist of prime numbers: {3, 11, 13, 17, 19, 23, 29}

5. In year 1 Python Programming course, we have implement simple Python program to generate Prime Number. Implement this Python program again and named as

prime_Standard.py. Compare these two programs and check which one is more efficient. Why?

7.6 Parsing a family tree

Now that we are more familiar with logic programming, let's use it to solve an interesting problem. Consider the following family tree:



As shown in this figure, John and Megan have three sons – William, David, and Adam.

The wives of William, David, and Adam are Emma, Olivia, and Lily respectively.

William and Emma have two children – Chris and Stephanie.

David and Olivia have five children – Wayne, Tiffany, Julie, Neil, and Peter.

Adam and Lily have one child – Sophia.

Based on these facts, we can create a program that can tell us the name of Wayne's grandfather or Sophia's uncles.

Even though we have not explicitly specified anything about the grandparent or uncle relationships, **logic programming** can **infer** them.

These relationships are specified in a file called **relationships.json** provided for you.

The file looks like the following:

```

{
    "father":
    [
        {"John": "William"},
        {"John": "David"},
        {"John": "Adam"},
        {"William": "Chris"},
        {"William": "Stephanie"},
        {"David": "Wayne"},
        {"David": "Tiffany"},
        {"David": "Julie"},
        {"David": "Neil"},
        {"David": "Peter"},
        {"Adam": "Sophia"}
    ],
    "mother":
    [
        {"Megan": "William"},
        {"Megan": "David"},
        {"Megan": "Adam"}
    ]
}

```

1. Create a new Python file and import the following packages:

```

import json
from kanren import Relation, facts, run, conde, var, eq

```

2. Define a function to check if x is the parent of y. We will use the logic that if x is the parent of y, then x is either the father or the mother. We have already defined "father" and "mother" in the fact base:

```

# Check if 'x' is the parent of 'y'
def parent(x, y):
    return conde([father(x, y)], [mother(x, y)])

```

3. Define a function to check if x is the grandparent of y. We will use the logic that if x is the grandparent of y, then the offspring of x will be the parent of y:

```

# Check if 'x' is the grandparent of 'y'
def grandparent(x, y):
    temp = var()
    return conde((parent(x, temp), parent(temp, y)))

```

4. Define a function to check if x is the sibling of y. We will use the logic that if x is the sibling of y, then x and y will have the same parents. Notice that there is a slight modification needed here because when we list out all the siblings of x, x will be listed as well because x satisfies these conditions. So, when we print the output, we will have to remove x from the list. We will discuss this in the main function:

```

# Check for sibling relationship between 'a' and 'b'
def sibling(x, y):
    temp = var()

```

```
return conde((parent(temp, x), parent(temp, y)))
```

5. Define a function to check if x is y's uncle. We will use the logic that if x is y's uncle, then x grandparents will be the same as y's parents. Notice that there is a slight modification needed here because when we list out all the uncles of x, x's father will be listed as well because x's father satisfies these conditions. So, when we print the output, we will have to remove x's father from the list. We will discuss this in the main function:

```
# Check if x is y's uncle
def uncle(x, y):
    temp = var()
    return conde((father(temp, x), grandparent(temp, y)))
```

6. Define the main function and initialize the relations father and mother:

```
if __name__ == '__main__':
    father = Relation()
    mother = Relation()
```

7. Load the data from the relationships.json file:

```
with open('relationships.json') as f:
    d = json.loads(f.read())
```

8. Read the data and add it to the fact base:

```
for item in d['father']:
    facts(father, (list(item.keys())[0], list(item.values())[0]))

for item in d['mother']:
    facts(mother, (list(item.keys())[0], list(item.values())[0]))
```

9. Define the variable x:

```
x = var()
```

10. We are now ready to ask some questions and see if the solver can come up with the right answers. Let's ask who John's children are:

```
# John's children
name = 'John'
output = run(0, x, father(name, x))
print("\nList of " + name + "'s children:")
for item in output:
    print(item)
```

11. Who is William's mother?

```
# William's mother
name = 'William'
output = run(0, x, mother(x, name))[0]
print("\n" + name + "'s mother:\n" + output)
```

12. Who are Adam's parents?

```
# Adam's parents
name = 'Adam'
output = run(0, x, parent(x, name))
print("\nList of " + name + "'s parents:")
for item in output:
    print(item)
```

13. Who are Wayne's grandparents?

```
# Wayne's grandparents
name = 'Wayne'
output = run(0, x, grandparent(x, name))
print("\nList of " + name + "'s grandparents:")
for item in output:
    print(item)
```



Workshop 7.3 : Logic Programming on Family Tree Parsing

1. Combine all the above code and named it as **family_tree.py**.
2. Finish the code by implement all the following queries:
 - Who is John's children?
 - Who is William's mother?
 - Who are Adam's parents?
 - Who are Wayne's grandparents?
 - Who are Megan's grandchildren?
 - Who are David's siblings?
 - Who are Tiffany's uncles?
 - List all the spouses in the family.

Your answer should be something like that:

List of John's children:

William

David

Adam

William's mother:

Megan

List of Adam's parents:

John

Megan

List of Wayne's grandparents:

Megan

John

List of Megan's grandchildren:

Stephanie

Neil

Sophia

Chris

Wayne

Julie

Peter

Tiffany

List of David's siblings:

William

Adam

List of Tiffany's uncles:

William

Adam

7.7 Analyzing geography

Let's use logic programming to build a solver to analyze geography.

In this problem, we will specify information about the location of various states in the US and then query the program to answer various questions based on those facts and rules.

The following is a map of the US:



1. Create a new Python file and import the following:

```
from kanren import run, fact, eq, Relation, var
```

2. Initialize the relations:

```
adjacent = Relation()
coastal = Relation()
```

3. Define the input files to load the data from:

```
file_coastal = 'coastal_states.txt'
file_adjacent = 'adjacent_states.txt'
```

4. Load the data:

```
# Read the file containing the coastal states
with open(file_coastal, 'r') as f:
    line = f.read()
    coastal_states = line.split(',')
```

5. Add the information to the fact base:

```
# Add the info to the fact base
for state in coastal_states:
    fact(coastal, state)
```

6. Read the adjacent data:

```
# Read the file containing the coastal states
with open(file_adjacent, 'r') as f:
    adjlist = [line.strip().split(',') for line in f if line and
line[0].isalpha()]
```

7. Add the adjacency information to the fact base:

```
# Add the info to the fact base
for L in adjlist:
    head, tail = L[0], L[1:]
    for state in tail:
        fact(adjacent, head, state)
```

8. Initialize the variables x and y:

```
# Initialize the variables
x = var()
y = var()
```

9. We are now ready to ask some questions. Check if Nevada is adjacent to Louisiana:

```
# Is Nevada adjacent to Louisiana?
output = run(0, x, adjacent('Nevada', 'Louisiana'))
print('\nIs Nevada adjacent to Louisiana?:')
print('Yes' if len(output) else 'No')
```

10. Print out all the states that are adjacent to Oregon:

```
# States adjacent to Oregon
output = run(0, x, adjacent('Oregon', x))
print('\nList of states adjacent to Oregon:')
for item in output:
    print(item)
```

11. List all the coastal states that are adjacent to Mississippi:

```
# States adjacent to Mississippi that are coastal
output = run(0, x, adjacent('Mississippi', x), coastal(x))
print('\nList of coastal states adjacent to Mississippi:')
for item in output:
    print(item)
```



Workshop 7.4 : Logic Programming on Analyzing Geography

1. Combine all the above code and named it as **geography_analyzer.py**.
2. Finish the code by implement all the following queries:
 - Is Nevada adjacent to Louisiana?

- List of states adjacent to Oregon.
- List of coastal states adjacent to Mississippi.
- List seven states that border a coastal state.
- List of states that are adjacent to Arkansas and Kentucky.

You results should be something like that:

```
Is Nevada adjacent to Louisiana?:
No
```

```
List of states adjacent to Oregon:
Nevada
Washington
Idaho
California
```

```
List of coastal states adjacent to Mississippi:
Louisiana
Alabama
```

```
List of 7 states that border a coastal state:
New York
Delaware
North Carolina
Arkansas
Pennsylvania
Alabama
Virginia
```

```
List of states that are adjacent to Arkansas and Kentucky:
Missouri
Tennessee
```



7.8 Mini-project: Building a Puzzle Solver

So far, we have learnt all the basic about Logic Programming. Let's try something more interesting - **Puzzle Solver**.

Another interesting application of logic programming is solving puzzles. We can specify the conditions of a puzzle and the program will come up with a solution.

In this mini-project, we will specify various bits and pieces of information about four people and ask for the missing piece of information.

In the logic program, we specify the puzzle as follows:

- Steve has a blue car.
- The person who owns a cat lives in Canada. Matthew lives in the USA.
- The person with a black car lives in Australia.
- Jack has a cat.
- Alfred lives in Australia.
- The person who has a dog lives in France.
- Who has a rabbit?

The goal is to find the person who has a rabbit.

Here are the full details about the four people:

	Pet	Car Color	Country
Steve	dog	blue	France
Jack	cat	green	Canada
Matthew	rabbit	yellow	USA
Alfred	parrot	black	Australia

Implement the **puzzle_solver.py** logic program by implementing the following steps (as Hints):

1. Create a new Python file and import the `kanren` and related packages.
2. Declare the variable `people`.
3. Define all the rules using **`lall`**. The first rule is that there are four people.
4. The person named Steve has a blue car.
5. The person who has a cat lives in Canada.
6. The person named Matthew lives in the USA.
7. The person who has a black car lives in Australia.
8. The person named Jack has a cat.
9. The person named Alfred lives in Australia.
10. The person who has a dog lives in France
11. One of the people in this group has a rabbit. Who is that person?
12. Run the solver with the preceding constraints.
13. Extract the output from the solution.
14. Print the full matrix obtained from the solver

If you run the code, it should be something like that:

Matthew is the owner of the rabbit

Here are all the details:

Name	Pet	Color	Country
Steve	dog	blue	France
Jack	cat	~_115	Canada

7.9 Summary

In this workshop, we learned how to write Python programs using logic programming.

We discussed how various programming paradigms deal with building programs.

We understood how programs are built in logic programming. We learned about various building blocks of logic programming and discussed how to solve problems in this domain. We implemented various Python programs to solve interesting problems and puzzles.

In the next chapter, we will learn about heuristic search techniques and use those algorithms to solve real-world problems.

In []: